



同濟大學
TONGJI UNIVERSITY

双自由度云台实时人体跟踪系统 设计与实现

综合设计与实践 B

课程大作业报告

授课教师： 龚炜 王晓年

小组成员： 梁育铭 张思琦 郑凯博

2026 年 6 月 15 日

摘要

本项目基于 Orange Pi 5 Pro(RK3588)开发板设计并实现了一套双自由度云台实时人体跟踪系统。系统以 USB 摄像头作为输入,支持 OpenCV Haar/HOG 轻量检测、YOLO ONNX CPU 推理和 YOLO RKNN NPU 推理三类目标检测路线;检测器统一输出目标框、中心点、类别与置信度,主程序据此计算目标中心相对画面中心的像素偏差,并通过比例控制算法驱动 PCA9685 舵机控制板,使 pan/tilt 两个舵机完成闭环跟踪。工程同时实现了 CSV 指标记录、Web 远程监控、目标出现语音提示、C 扩展加速、pypm 并行加速和 RKNN/NPU 部署验证。

系统运行过程中记录了 1296 帧 RKNN 云台跟踪日志,其中 1096 帧为有效新鲜目标帧,有效帧率为 84.6%,全程有效帧平均像素误差为 156.59 px;选取第 855–954 帧作为稳定片段时,100/100 帧均为有效目标帧,平均像素误差为 81.78 px,中位误差为 70.30 px。系统还完成了 Web 监控、语音提示、逐帧指标记录、C 扩展卷积与 NMS、pypm 并行卷积,以及 RKNN/NPU 板端部署等工作。

关键词: 双自由度云台; 人体检测; YOLO; RKNN; PCA9685; C 扩展; pypm; Web 监控

目录

第 1 章 项目背景与任务要求	1
1.1 课题目标	1
1.2 实现内容概览	1
1.3 设计难点	2
第 2 章 系统总体方案	2
2.1 闭环工作流程	2
2.2 功能模块划分	2
2.3 检测方案与部署环境	3
2.4 系统组织方式	3
第 3 章 目标识别模块设计	3
3.1 检测任务与方案选择	3
3.2 YOLO11n 网络结构	4
3.3 统一输出与后处理	4
3.4 主目标选择与检测效果	5
第 4 章 舵机控制与云台跟踪设计	5
4.1 PCA9685 舵机驱动	5
4.2 角度映射与安全限位	6
4.3 比例控制策略	6
4.4 运行校准与保护	6
第 5 章 系统整合与功能实现	7
5.1 主循环整合	7
5.2 运行模式与演示场景	7
5.3 Web 监控功能	7
5.4 语音提示与指标记录	8
第 6 章 扩展功能与性能优化	9
6.1 C/C++ 加速任务	9
6.2 NMS 后处理加速	9
6.3 pypm 并行实验	10
6.4 RKNN NPU 部署	11
第 7 章 实验结果与分析	11
7.1 评价指标与数据依据	11
7.2 跟踪与检测性能	12
7.3 真实云台运行结果	13

7.4 逐帧误差统计	13
第 8 章 小组分工	14
8.1 分工原则	14
8.2 成员职责	14
第 9 章 问题与改进	15
9.1 当前不足	15
9.2 后续改进方向	15
第 10 章 总结	15
10.1 项目完成情况	15
10.2 总体评价	15
参考文献	16
附录 A 项目链接	17
附录 B 核心代码	17
B1 主程序闭环流程	17
B2 云台比例控制	18
B3 YOLO 后处理中的 C NMS 加速入口	19

第 1 章 项目背景与任务要求

1.1 课题目标

本课程设计要求在开发板上完成一个具有实际运行能力的目标跟踪系统，并使用两自由度云台完成目标跟随。对于云台跟踪任务，系统不仅需要能够识别目标，还需要在嵌入式平台上稳定运行，并通过云台运动把目标尽量保持在画面中心。因此，本项目围绕“实时人体识别”和“双自由度云台闭环控制”两个核心问题展开。

本项目最终实现了一套基于 Orange Pi 5 Pro 开发板的实时人体跟踪系统。系统通过 USB 摄像头采集视频画面，利用 OpenCV 或 YOLO 模型检测人体目标，根据目标中心与画面中心之间的偏差计算云台控制量，再由 PCA9685 驱动 pan、tilt 两个舵机完成姿态调整。系统还加入了 Web 远程监控、语音提示、性能记录、C 扩展加速、pymmp 并行加速和 RKNN/NPU 部署验证，使整机具备演示、运行记录和实验分析能力。

1.2 实现内容概览

系统功能	本项目实现
开发板部署运行	在 Orange Pi 5 Pro / RK3588 平台上完成摄像头采集、目标识别、云台控制和监控展示。
两自由度云台	使用 PCA9685 舵机驱动板控制 pan、tilt 两个舵机，实现水平和垂直方向跟踪。
跟踪速度与稳定性	记录检测帧率、推理耗时、舵机角度、目标丢失情况和真实云台运行状态。
C/C++ 扩展	使用 C 扩展实现 5x5 卷积和 NMS 加速，并与 Python 实现进行性能对比。
网络服务	实现浏览器 Web 监控页面，显示实时画面、目标状态、误差和云台角度。
pymmp 加速	使用 pymmp 对卷积任务进行多 worker 并行对比实验。
RKNN 部署	将 YOLO 模型部署到 RK3588 NPU，并完成板端推理验证。

表 1: 系统功能与实现对应关系

1.3 设计难点

与单纯的离线识别任务不同，本项目的难点在于完整闭环：检测结果必须及时转化为云台运动，云台运动又会改变摄像头画面，进而影响下一帧检测。系统设计时需要同时考虑识别效果、实时性、舵机安全范围和开发板资源限制。

第 2 章 系统总体方案

2.1 闭环工作流程

系统整体采用“感知、决策、执行、反馈”的闭环结构。感知部分由摄像头和目标识别模块完成，负责从画面中获得人体目标的位置；决策部分由控制器完成，负责把像素偏差转换为云台角度修正；执行部分由 PCA9685 和两个舵机完成，负责实际转动云台；反馈部分由下一帧摄像头画面、日志记录和 Web 监控共同构成。

每一帧的处理过程如下：摄像头采集当前画面后，目标检测模块输出目标边界框和中心点；系统计算目标中心与画面中心的水平、垂直偏差；控制器根据比例控制策略生成 pan、tilt 两个方向的角度步进；舵机驱动模块将角度转换为 PWM 信号并控制云台；同时，系统记录当前帧的检测状态、帧率、误差、舵机角度和耗时数据，并可在浏览器端实时显示。

2.2 功能模块划分

功能模块	主要作用
视频采集模块	从 USB 摄像头获取实时图像，并为后续检测提供输入。
目标识别模块	支持 OpenCV 轻量检测和 YOLO 人体检测，输出目标位置与置信度。
云台控制模块	根据目标偏差计算舵机角度修正，实现 pan/tilt 闭环跟踪。
舵机驱动模块	通过 PCA9685 输出 PWM 信号，控制两个舵机在安全范围内运动。
Web 监控模块	在浏览器中显示实时画面、目标锁定状态、误差和舵机角度。
语音提示模块	在目标首次出现时播放提示音，提高系统交互性。
性能优化模块	通过 C 扩展、pymp 和 RKNN 部署提升关键环节运行效率。

表 2: 系统主要功能模块

2.3 检测方案与部署环境

本系统保留了多种检测路线。OpenCV 检测作为低开销保底方案，适合硬件资源紧张或现场演示时快速验证；YOLO ONNX 路线作为 CPU 上的 DNN 对照方案；YOLO RKNN 路线作为最终的 NPU 部署方案。不同检测路线的输出统一为目标位置和目标状态，因此云台控制、监控展示和指标记录可以复用同一套闭环逻辑。

项目	配置
开发板	Orange Pi 5 Pro, RK3588/RK3588S, 约 4 GB 内存。
摄像头	USB UVC 摄像头，采集分辨率 640×480。
云台与驱动	PCA9685 舵机驱动板，I2C bus 1，地址 0x40，pan 通道 0，tilt 通道 1。
Python 环境	Python 3.10，包含 OpenCV、NumPy、smbus2、onnxruntime、rknnlite 和 pypm 等依赖。
NPU 环境	RKNPU 驱动 v0.9.6，RKNN runtime/toolkit 2.3.2，主要使用轻量级 YOLO11n RKNN 模型。

表 3: 硬件平台与软件环境

2.4 系统组织方式

从系统组织上看，最终实现按照视频采集、目标识别、闭环控制、舵机驱动、监控交互和性能优化等职责拆分。各模块通过统一的数据结构和运行参数连接，既便于切换不同检测方案，也便于在硬件调试、性能测试和最终演示之间复用同一套主流程。

第 3 章 目标识别模块设计

3.1 检测任务与方案选择

目标识别模块的任务是在实时视频帧中找到需要跟踪的人体目标，并输出目标框、类别、置信度和中心位置。系统最终以人体作为主要跟踪对象，同时保留人脸检测作为近距离场景下的稳定保底方案。

OpenCV 方案主要使用 Haar 级联与 HOG 行人检测方法。Haar 级联检测具有实现简单、运行速度快的特点，适合对近距离人脸进行快速定位；HOG 行人检测则能够在不依赖深度模型的情况下提供人体检测能力。两者共同构成了系统的轻量级检测路线。Haar 级联与 HOG 行人检测分别来自经典目标检测方法^[1,2]，在嵌入式平台上具有较好

的可用性。

DNN 方案采用 YOLO11n 作为人体检测模型，跟踪 COCO 数据集中的人体类别^[3]。YOLO 系列模型将目标定位和分类统一到单阶段检测框架中，具有较好的实时性^[4,5]。本项目选择轻量版本的 YOLO11n，是因为开发板内存和算力有限，模型需要在准确率和推理速度之间取得平衡。

3.2 YOLO11n 网络结构

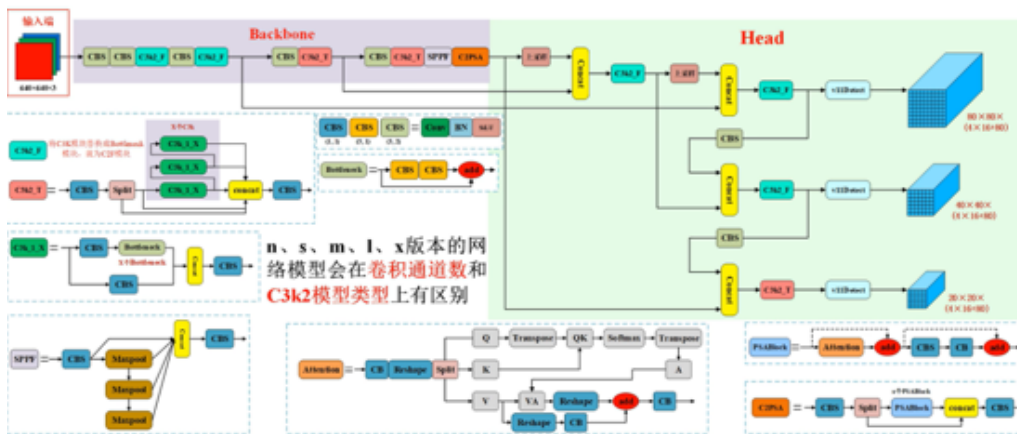


图 1: YOLO11n 网络结构示意图

图 1 展示了 YOLO11n 的整体结构。其骨干网络负责提取多层视觉特征，颈部结构负责融合不同尺度信息，检测头则输出目标框和类别概率。对于云台跟踪任务而言，模型不仅要识别是否存在人体，还要稳定输出目标中心位置，供后续控制模块使用。

3.3 统一输出与后处理

为了降低整合复杂度，系统把 OpenCV、ONNX 和 RKNN 后端统一为同一种目标结果格式，其核心信息包括目标中心点、边界框、类别、置信度和是否为历史保持目标的标志。OpenCV 路线在漏检后的短时间内会保留上一目标并进行标记；YOLO ONNX 与 YOLO RKNN 路线也采用相同的保持策略。主循环只根据目标是否存在以及是否为历史结果决定是否更新云台，因此不同检测器之间可以通过运行参数切换。

YOLO 后处理包括 letterbox 坐标还原、YOLOv5 与 YOLOv8/YOLO11 输出形状兼容、置信度阈值筛选、类别筛选、NMS 和主目标选择。ONNX 路线输入为 NCHW float32 RGB 张量，像素值归一化到 $[0, 1]$ ；RKNN 路线输入为 NHWC uint8 RGB 图像，归一化在模型转换参数中完成。RKNN 检测器若缺少运行环境或模型，会直接给出错误信息，不静默回退到 CPU，从而保证 NPU 部署实验结果可验证。

3.4 主目标选择与检测效果

当画面中存在多个候选目标时，系统采用“面积优先、中心优先”的主目标选择策略。面积较大的目标通常距离摄像头更近，画面中心附近的目標則更符合当前跟踪意图。综合这两个因素，可以减少多人场景下云台频繁切换目标的问题。



图 2: YOLO 人体检测效果示例

图 2 为人体检测效果示例。可以看到，模型能够在室内背景下输出人体目标框和较高置信度，为后续云台控制提供了稳定输入。

第 4 章 舵机控制与云台跟踪设计

4.1 PCA9685 舵机驱动

云台控制部分的目标是将图像坐标系中的目标偏差转换为两个舵机的运动。系统使用 PCA9685 舵机驱动板控制 pan 和 tilt 两个方向。PCA9685 是 16 通道 12 位 PWM 控制器，可通过 I2C 输出舵机控制脉冲^[6]，适合同时驱动多个舵机。

本项目默认使用 I2C bus 1 和地址 0x40 连接 PCA9685，pan 轴接通道 0，tilt 轴接通道 1，PWM 频率设为 50 Hz。驱动初始化时先使 PCA9685 进入 sleep 状态，写入由 25 MHz 振荡器和 4096 计数周期计算得到的 PRESCALE 分频值，再退出 sleep 并开启自动地址递增。实际输出 PWM 时，程序向目标通道对应的 4 个寄存器写入起始计数和结束计数；本项目令起始计数为 0，主要由结束计数控制高电平脉宽。

4.2 角度映射与安全限位

舵机控制信号本质上是周期约 20 ms 的 PWM 脉冲，不同脉宽对应不同舵机角度。系统采用线性映射将目标角度转换为 PWM 计数值：

$$p = p_{\min} + \frac{\theta}{180^\circ}(p_{\max} - p_{\min}), \quad (1)$$

其中 θ 表示目标角度， $p_{\min}$ 和 $p_{\max}$ 分别表示舵机可接受的最小和最大脉宽计数。为避免机械撞限，系统对 pan 和 tilt 两个轴分别设置安全角度范围，并在每次输出前进行限幅。

4.3 比例控制策略

闭环控制采用比例控制思想。设图像宽高为 (W, H) ，目标中心为 (x_t, y_t) ，画面中心为 $(W/2, H/2)$ ，则像素误差为

$$e_x = x_t - \frac{W}{2}, \quad e_y = y_t - \frac{H}{2}. \quad (2)$$

当误差位于死区范围内时，系统认为目标已经基本居中，不再移动对应舵机；当误差超过死区时，系统按比例增益计算角度修正，并限制单帧最大角度变化，避免云台抖动或突然大幅转动。

控制器实际使用的步进形式为

$$\Delta\theta_{pan} = \text{clip}(s_{pan}k_{pan}e_x, -m, m), \quad \Delta\theta_{tilt} = \text{clip}(s_{tilt}k_{tilt}e_y, -m, m), \quad (3)$$

其中 k_{pan} 、 k_{tilt} 为比例增益， s_{pan} 、 s_{tilt} 为方向符号， m 为单帧最大角度步长。实验中水平和垂直比例增益均设为 0.015，死区设为 25 像素，单帧最大角度步长设为 2.0 度。常规安全范围为 pan 45–135 度、tilt 60–120 度，最终硬件演示中进一步使用保守的 tilt 中心角和范围，以适配实际云台机械结构。

4.4 运行校准与保护

比例控制的优点是结构简单、实时性好、参数易调。对于本项目的两自由度小型云台而言，比例控制已经能够完成目标居中和方向跟随。真实运行时，系统通过调整水平和垂直方向的符号，使舵机运动方向与摄像头成像方向一致；若目标丢失，则保持当前角度不变，避免云台被旧目标位置继续牵引。

云台安全是本项目实现中的重点。系统在真实运行前先进行小幅校准，确认两个舵机的方向、中心角和机械范围；运行过程中所有目标角度都经过限位处理；最终演示中还使用较保守的 tilt 范围和最大步长，降低开机或目标剧烈移动时的撞限风险。

第 5 章 系统整合与功能实现

5.1 主循环整合

在系统整合阶段，需要把目标识别、云台控制、状态记录和展示交互连接成完整闭环。主循环按照固定顺序处理每一帧：采集图像、检测目标、计算控制量、更新舵机、记录指标、刷新展示状态。为了便于不同检测后端切换，系统将 OpenCV、ONNX 和 RKNN 的输出统一为目标位置和目标状态，后续控制逻辑不再关心检测器内部实现。

5.2 运行模式与演示场景

系统运行时可根据实验目的选择不同模式。调试阶段通常使用模拟舵机和无显示模式快速验证主循环；真实演示阶段启用摄像头、PCA9685 舵机驱动、目标检测、Web 监控和语音提示；性能测试阶段则固定检测后端和运行帧数，记录每帧耗时与目标状态。这样的配置方式既能保护硬件，也能保证各项实验结果具有可复现性。

为了便于现场演示，系统预设了多种运行场景，分别覆盖最终整合演示、RKNN 跟踪、Web 监控、语音提示验证和单轴舵机测试等需求。每种场景对应固定的功能组合和日志记录方式，使演示流程更稳定。

5.3 Web 监控功能

Web 监控是系统的重要展示功能。它不单独占用摄像头，而是由主循环把当前画面和状态推送给 Web 服务，浏览器端再实时显示目标框、画面中心、帧率、误差、舵机角度和目标锁定状态。这样既便于远程观察，也不会破坏采集和控制的实时链路。



图 3: Web 监控截图

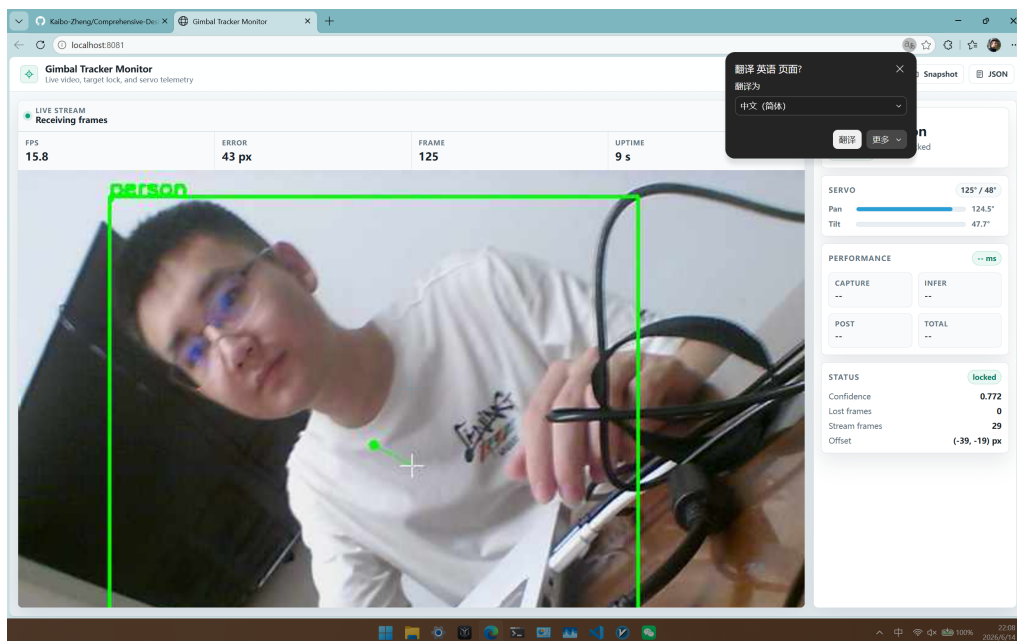


图 4: 浏览器中的监控页面

5.4 语音提示与指标记录

语音提示用于增强系统交互性。当目标从未检测到变为首次检测到时，系统播放一次提示音；之后在冷却时间内不重复播报，避免连续检测时频繁播放。考虑到开发板内

存有限，最终演示优先使用预生成中文音频，而不是在跟踪过程中实时加载大型语音合成模型。

系统还记录每帧的运行指标，包括帧率、目标是否检出、目标偏差、舵机角度、采集耗时、推理耗时、后处理耗时、总耗时和连续丢失帧数等。统一的日志格式使 OpenCV、ONNX 和 RKNN 路线能够直接对比，也为后续统计跟踪误差和定位性能瓶颈提供了依据。

第 6 章 扩展功能与性能优化

在完成基本跟踪闭环后，系统进一步实现了 C/C++ 加速、pymv 并行加速和 RKNN/NPU 部署，用于比较不同实现方式对运行效率的影响。

6.1 C/C++ 加速任务

C/C++ 加速部分选择 5x5 图像卷积作为对照任务。该任务计算规则清晰，便于比较 Python 实现和 C 扩展实现的性能差异。C 扩展实现了卷积、NMS 和 Python 模块接口^[7]，并以 Python 可调模块形式集成到系统中。实验中，Python/NumPy 参考实现耗时约 1493.016 ms/frame，C 扩展耗时约 0.915 ms/frame，加速比约 1632 倍，输出最大差异约为 4.6×10^{-5} ，说明 C 扩展在保证数值一致性的同时显著降低了计算耗时。

6.2 NMS 后处理加速

除卷积外，系统还将 YOLO 后处理中的 NMS 纳入 C 扩展加速。NMS 是目标检测后处理中常见的耗时步骤，用于去除重叠候选框。实验中，Python NMS 平均耗时约 52.463 ms/run，C NMS 平均耗时约 7.093 ms/run，加速比约 7.40 倍。运行时系统会优先使用 C 扩展，若扩展不可用则自动回退到 Python NMS，保证功能完整性。

任务	基准	最终结果	效果
5x5 卷积	Python/NumPy 1493.016 ms/frame	C 扩展 0.915 ms/frame	约 1632 倍加速，最大输出差异 4.6×10^{-5}
5x5 卷积	Python/NumPy 1493.016 ms/frame	pypm x4 447.617 ms/frame	约 3.34 倍加速
YOLO NMS	Python NMS 52.463 ms/run	C NMS 7.093 ms/run	约 7.40 倍加速，保留索引与 Python 结果一致

表 4: 加速模块性能结果

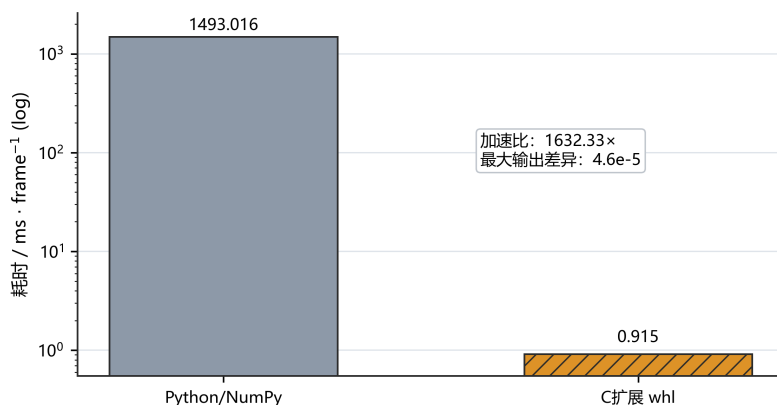


图 5: Python、pypm 与 C 扩展 5x5 卷积耗时对比

6.3 pypm 并行实验

pypm 加速实验同样以 5x5 卷积为对象^[8]。课程结果汇总表采用与 C 扩展相同输入规模下的对比口径，4 worker 情况下相对 Python/NumPy 参考实现取得约 3.34 倍加速。另一次 worker 扫描实验还记录了 x1、x2、x4、x8 等不同 worker 数的变化趋势，图 6 用于展示并行度增加后耗时下降的趋势。该结果说明，对于显式循环和按行可拆分的任务，多进程并行可以有效提升运行速度。

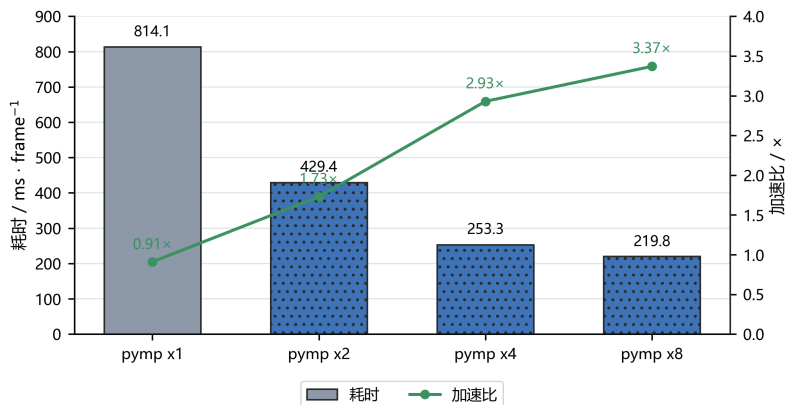


图 6: pypmp worker 数与加速比

6.4 RKNN NPU 部署

RKNN 部署部分将 YOLO 模型转换为适合 RK3588 NPU 运行的格式，并在开发板上完成推理验证^[9]。转换过程设置目标平台、输入输出格式和模型归一化参数，最终得到可由 RKNN runtime 加载的板端模型。ONNX CPU 路线保留为对照^[10]，RKNN NPU 路线作为最终部署路线。早期短时验证中，YOLO ONNX CPU 平均推理耗时约 137 ms，YOLO RKNN NPU 平均推理耗时约 71.36 ms；在 1296 帧真实 RKNN 云台跟踪日志中，平均推理耗时约 53.7 ms，平均运行速度约 14.7 FPS。本文实验分析以逐帧跟踪日志作为精度评价依据，并保留 ONNX CPU 数据作为部署前后的速度对照。

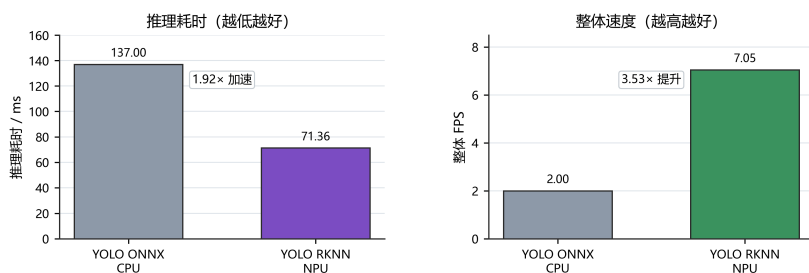


图 7: YOLO ONNX CPU 与 RKNN NPU 速度对比

第 7 章 实验结果与分析

7.1 评价指标与数据依据

实验从检测速度、真实云台运行稳定性、逐帧像素误差和扩展功能加速效果四个方面进行评价。本文采用最终汇总表、跟踪性能记录、加速性能记录、板端环境记录以及

RKNN 真实运行逐帧日志作为实验依据。逐帧日志共记录 1296 帧，包含目标检出状态、水平误差、垂直误差、合成像素误差、舵机角度、帧率和各阶段耗时，因此可以进一步统计平均像素误差、最大像素误差和 95 分位误差。

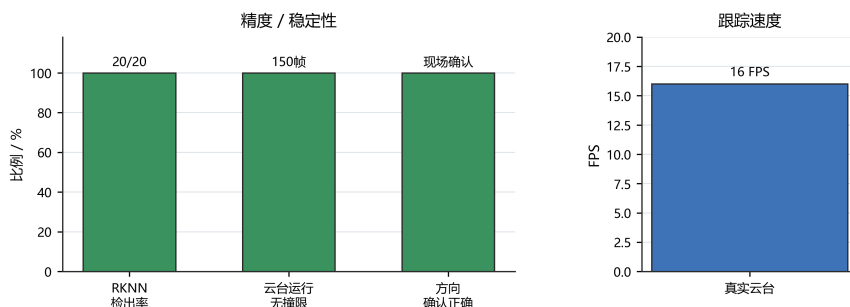


图 8: 保留的跟踪评价指标与真实云台速度

7.2 跟踪与检测性能

模式	检测器	设备	平均推理耗时	结果说明
OpenCV 保底方案	人脸检测	CPU	约 63 ms	约 9.3 FPS，轻量保底路线。
YOLO ONNX	YOLO11n 人体检测	CPU	约 137 ms	约 2.0 FPS，DNN CPU 对照路线。
YOLO RKNN	YOLO11n 人体检测	RK3588 NPU	约 53.7 ms	1296 帧真实运行日志，平均约 14.7 FPS。
真实云台跟踪	RKNN 检测与双轴控制	PCA9685 舵机	—	有效目标帧 1096 帧，稳定片段 100/100 帧有效。

表 5: 跟踪与检测性能汇总

从结果看，OpenCV 人脸检测方案虽然检测能力较简单，但运行稳定、速度较高，适合作为现场演示保底方案；YOLO ONNX CPU 路线能够验证 DNN 检测功能，但在开发板 CPU 上推理耗时较高；YOLO RKNN NPU 路线明显降低了推理耗时，并在真实云台运行中形成了完整逐帧日志，说明 DNN 部署已经能够参与闭环跟踪。

7.3 真实云台运行结果

真实云台运行结果显示，系统完成了 1296 帧 RKNN 跟踪日志记录，其中有效新鲜目标帧为 1096 帧，有效帧率为 84.6%，平均运行速度约为 14.7 FPS。运行过程中日志持续记录 pan/tilt 舵机角度、目标偏差和各阶段耗时，说明检测、控制和指标记录链路能够连续工作。

7.4 逐帧误差统计

从完整日志看，全程有效帧平均像素误差为 156.59 px，中位误差为 153.01 px，95 分位误差为 282.65 px。该结果反映了目标移动、云台初始响应、短时目标丢失和机械调整共同作用下的整体误差水平。为了评价系统在目标稳定进入视野且云台完成初步响应后的跟踪能力，进一步选取第 855–954 帧作为稳定跟踪片段。该片段连续 100 帧均为有效目标帧，平均像素误差为 81.78 px，中位误差为 70.30 px，95 分位误差为 184.11 px，且 83% 的有效帧误差不超过 150 px。

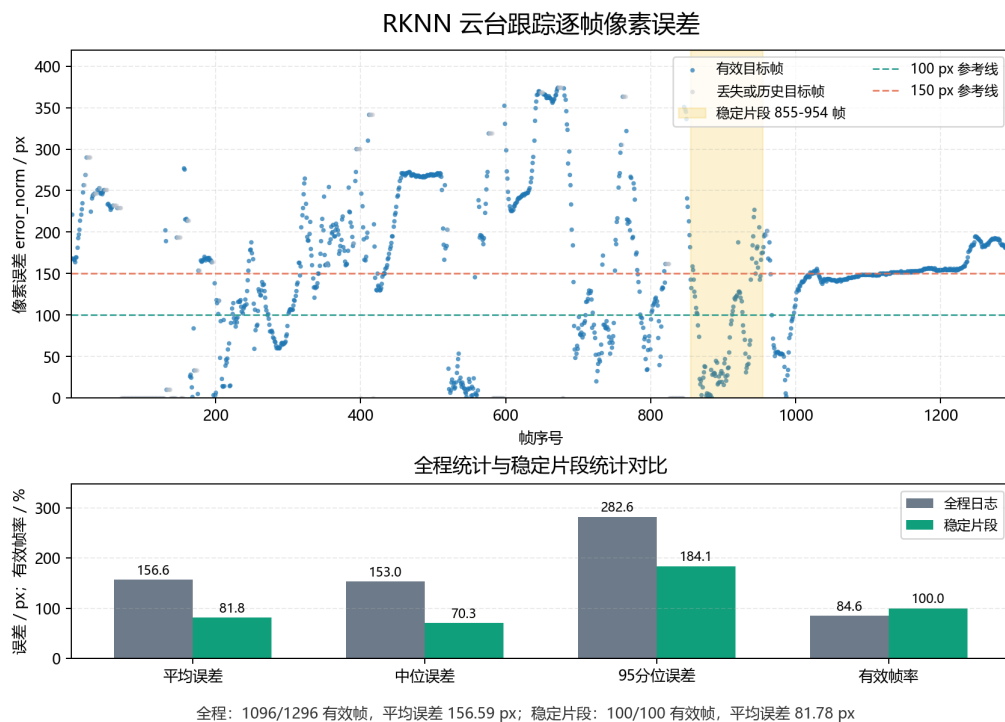


图 9: RKNN 云台跟踪逐帧像素误差与稳定片段统计

统计范围	有效帧率	平均误差	中位误差	95 分位误差	说明
全程日志	84.6%	156.59 px	153.01 px	282.65 px	反映完整运行过程中的整体误差。
稳定片段	100.0%	81.78 px	70.30 px	184.11 px	反映目标稳定进入视野后的跟踪能力。

表 6: RKNN 逐帧像素误差统计

第 8 章 小组分工

8.1 分工原则

本项目由三名成员协作完成，整体分为目标识别、舵机控制和系统整合优化三个方向。每位成员负责本人方向的技术实现、实验整理、报告材料和 PPT 内容，最终再统一整合为双自由度云台人体跟踪系统。

8.2 成员职责

成员	负责方向	具体工作
梁育铭	模型训练与目标识别	负责人体目标识别方案、YOLO 模型训练与检测效果验证，整理目标检测相关原理、模型结构和识别结果，并完成本人负责部分的报告和 PPT 内容。
张思琦	舵机控制与云台执行	负责 PCA9685 舵机驱动、pan/tilt 双自由度云台控制、角度到 PWM 的映射、安全限位和硬件调试，整理舵机控制相关原理与实验材料，并完成本人负责部分的报告和 PPT 内容。
郑凯博	系统整合与扩展优化	负责主程序整合、检测器接口统一、闭环跟踪流程、Web 远程监控、语音提示、CSV 指标记录、C 扩展加速、pypm 并行对比、RKNN/NPU 部署验证、实验结果汇总，以及本人负责部分的报告和 PPT 内容。

表 7: 小组成员分工

第 9 章 问题与改进

9.1 当前不足

当前系统仍有以下不足。第一，全程逐帧像素误差仍存在一定波动，完整日志的平均误差高于稳定片段结果，说明目标运动、机械响应和控制参数仍会影响居中精度。第二，短时 FPS 容易受到摄像头首帧暖机和采集阻塞影响，正式测试应采用更长时间运行并丢弃前若干帧。第三，RKNN 方案虽然已经跑通，但仍可通过 INT8 量化、更小输入尺寸或流水线并行进一步优化。第四，当前控制器为比例控制，后续可加入 PID、目标丢失搜索策略和多目标跟踪 ID，以提升复杂场景下的稳定性。

9.2 后续改进方向

后续改进可以从三个方向展开：一是完善数据分析脚本，自动划分启动段、稳定段和目标丢失段，并生成误差分布；二是优化实时性，将采集、推理和控制过程进一步解耦；三是增强控制策略，使云台运动更加平滑，并提高多人场景下目标选择的连续性。

第 10 章 总结

10.1 项目完成情况

本项目完成了基于 Orange Pi 5 Pro / RK3588 开发板的双自由度云台实时人体跟踪系统。系统实现了摄像头采集、OpenCV/YOLO 检测、PCA9685 舵机控制、Web 监控、语音提示、CSV 指标记录、C 扩展卷积与 NMS、pypm 并行卷积和 RKNN/NPU 部署。真实云台运行记录包含 1296 帧 RKNN 跟踪日志，其中有效新鲜目标帧为 1096 帧；稳定跟踪片段中连续 100 帧均为有效目标帧，平均像素误差为 81.78 px。

10.2 总体评价

整体来看，本项目完成了从摄像头采集、人体识别、云台控制到监控展示和性能记录的完整链路，并实现了 C/C++ 加速、Web 监控、pypm 并行和 RKNN 部署等功能。系统的核心价值在于将“人体识别”和“两自由度云台控制”连接成实时闭环，并在开发板资源有限的条件下完成可演示的整机系统。

参考文献

- [1] Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2001.
- [2] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 886–893, 2005.
- [3] Tsung-Yi Lin, Michael Maire, Serge Belongie, et al. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision*, pages 740–755, 2014.
- [4] Joseph Redmon and Ali Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [5] Ultralytics. YOLO11: Real-time object detection models. <https://docs.ultralytics.com/models/yolo11/>, 2024.
- [6] NXP Semiconductors. PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller. <https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf>, 2015.
- [7] Python Software Foundation. Extending and embedding the Python interpreter. <https://docs.python.org/3/extending/>, 2024.
- [8] Tim Meinhardt. pypm: Easy openmp style multiprocessing for python. <https://github.com/classner/pypm>, 2017.
- [9] Rockchip. RKNN Toolkit2 and RKNN runtime documentation. <https://github.com/airockchip/rknn-toolkit2>, 2024.
- [10] Microsoft. ONNX Runtime: Cross-platform, high performance scoring engine for ML models. In *Proceedings of Machine Learning and Systems*, 2020. URL <https://onnxruntime.ai/>.

附录 A 项目链接

项目代码托管地址：

<https://github.com/Kaibo-Zheng/Comprehensive-Design-and-Practice-B>

项目资料百度网盘链接：

链接：待补充

提取码：待补充

附录 B 核心代码

B1 主程序闭环流程

以下代码摘自 `tracker/app.py`，展示了每帧中摄像头采集、目标检测、控制计算、舵机更新和指标记录之间的关系。

```
while True:
    frame_start = time.perf_counter()
    ok, frame = cap.read()
    if not ok:
        raise RuntimeError("Failed to read frame from camera")

    target = detector.detect(frame)
    if voice is not None:
        voice.update(target)

    control_center = target.center if target is not None and not target.stale
    else None
    result = controller.update(control_center, frame_size=(frame.shape[1],
frame.shape[0]))

    if result.pan_moved:
        gimbal.set_angle("pan", result.pan)
    if result.tilt_moved:
        gimbal.set_angle("tilt", result.tilt)

    snapshot = metrics.update(
        found=target is not None,
        stale=bool(target and target.stale),
```

```
x_error=result.x_error,  
y_error=result.y_error,  
pan=result.pan,  
tilt=result.tilt,  
label=(target.label if target is not None else ""),  
confidence=(target.confidence if target is not None else None),  
timings=timings,  
)
```

B2 云台比例控制

以下代码摘自 motion/controller.py，展示了系统如何将目标中心相对画面中心的像素误差转换为 pan/tilt 两个方向的角度步进。

```
def update(self, target_center, frame_size):  
    if target_center is None:  
        return ControlResult(self.pan, self.tilt, 0, 0, False)  
  
    width, height = frame_size  
    x_error = int(target_center[0] - width / 2)  
    y_error = int(target_center[1] - height / 2)  
  
    pan_step = self._axis_step(  
        error=x_error,  
        gain=self.controller_config.pan_gain,  
        sign=self.controller_config.pan_sign,  
    )  
    tilt_step = self._axis_step(  
        error=y_error,  
        gain=self.controller_config.tilt_gain,  
        sign=self.controller_config.tilt_sign,  
    )  
  
    self.pan = clamp(self.pan + pan_step,  
                     self.servo_config.pan.min_angle,  
                     self.servo_config.pan.max_angle)  
    self.tilt = clamp(self.tilt + tilt_step,  
                      self.servo_config.tilt.min_angle,  
                      self.servo_config.tilt.max_angle)
```

B3 YOLO 后处理中的 C NMS 加速入口

以下代码摘自 `inference/postprocess.py`。系统会优先尝试调用 C 扩展实现的 NMS；当扩展不可用时，回退到 Python/Numpy 版本，保证检测流程仍可运行。

```
def nms(boxes_xyxy, scores, iou_threshold=0.45):
    if len(boxes_xyxy) == 0:
        return []

    boxes = np.asarray(boxes_xyxy, dtype=np.float64).reshape(-1, 4)
    scores_arr = np.asarray(scores, dtype=np.float64).reshape(-1)

    keep_c = _nms_c(boxes, scores_arr, iou_threshold)
    if keep_c is not None:
        return keep_c

    order = scores_arr.argsort()[::-1]
    keep = []
    while order.size > 0:
        i = int(order[0])
        keep.append(i)
        if order.size == 1:
            break
        rest = order[1:]
        ious = iou_batch(boxes[rest], boxes[i])
        order = rest[ious <= iou_threshold]
    return keep
```